Big Data: Systems, Programming & Management — A-Grade Exam Notes

Course: COMPSCI4064 (H) / COMPSCI5088 (M), University of Glasgow **Lecturer**: Dr. Richard McCreadie **Exam Format**: 1.5 hours (H-level), ~60-75 marks, 3 questions (all compulsory)

TABLE OF CONTENTS

---

1. Big Data Fundamentals (Week 1)

1.1 What is Big Data?

Definition: "Data whose capturing, storage, curation, management, and processing are not possible with traditional tools within a tolerable amount of time."

Big Data is defined by: - Its **Use** — how we store, access, and process it - The **Software and Hardware** available — "traditional tools" (software) and "tolerable amount of time" (hardware)

Big Data is **relative** — what counts as "big" changes over time as hardware improves. In the 90s, 100MB was big data; today 1PB might be.

1.2 Dimensions of Big Data (The 5 V's)

Dimension	Description	Exam Tip
Volume	Sheer size of data	Most obvious; always mention first
Variety	Structured, semi-structured, unstructured (text, video, audio, photos)	Mention data types in scenario
Velocity	Speed of data arrival; near-real-time requirements	Links to batch vs stream
Veracity	Trustworthiness — errors, alterations, noise	Often overlooked; easy marks
Variability	Data flow patterns — peaks and valleys, complexity of relationships	Mention when workload changes over time

> **Exam Pattern**: "Pick any two dimensions and you are probably working in a Big Data space." When identifying V's in a scenario, state the dimension [1 mark] then explain WHY it applies [1 mark].

1.3 Hardware Primer

Four key hardware aspects:

1. **General Purpose Compute (CPU)**: Speed measured by benchmarks, not cores/clock speeds. Limited by transistor density and reticle limit.
2. **Specialist Compute**: GPUs for deep learning, quantum cores. Used when CPU not optimised enough.
3. **Storage Stack** (fastest → slowest, smallest → largest):
4. Registers → Cache (L1-3) → Main Memory (RAM) → SSD → HDD
5. Upper tiers: small, expensive, fast, transient
6. Lower tiers: large, cheap, slow, persistent
7. **Network**: LAN transfer latency ~1ms. Network transfers nearly always slower than local storage.

Key latency comparisons (from Gregg):

Operation	Latency	Analogy (1 CPU cycle = 1 sec)
L1 cache	~1 ns	1 second
L2 cache	~4 ns	4 seconds
RAM	~100 ns	~2 minutes
SSD	~100 µs	~1 day
HDD	~10 ms	~4 months
LAN	~1 ms	~1 year

1.4 User Needs — Three Big Data Use Cases

Use Case	Description	Example
Extraction/Filtering	Extract subset matching criteria	SQL SELECT, batch search
Transformation	Apply function to each item	Search indexing, sentiment analysis
Aggregation	Merge and transform multiple items	Word counting, SQL JOIN, clustering

1.5 Batch vs Stream Processing

Aspect	Batch Processing	Stream Processing
Data type	Largely static	Continuously arriving
Processing	Large blocks analysed at once	Items processed individually or in micro-batches
Metric	**Completion time**	**Processing latency per item**
Example	ML training, search indexing	Fraud detection, live recommendations

1.6 Vertical vs Horizontal Scaling

Aspect	Vertical Scaling	Horizontal Scaling
Approach	Buy bigger machines (mainframes)	Add more commodity machines
Cost	Expensive; diminishing returns	Cost-effective; commodity hardware
Limits	Physical (reticle limit, transistor size)	Theoretically infinite
Complexity	Simple programming	Requires specialised tools & different programming paradigm
This course	Not the focus	**The entire focus of this course**

> **Key insight**: "The Big Data course is really a course about Horizontal Scaling of Compute and Storage, along with the tools that enable both."

1.7 Compute Parallelism

Definition: Solving a Big Data problem by breaking it into multiple tasks that can be solved simultaneously.

Where parallelism happens: 1. Within a CPU core (pipelining, superscalar) — invisible to programmer 2. Across CPU cores (threads) 3. **Across machines** — horizontal scaling (course focus)

Flynn's Taxonomy:

	Single Data (SD)	Multiple Data (MD)
Single Instruction (SI)	SISD (classic serial)	SIMD (GPUs, AVX)
Multiple Instruction (MI)	MISD (rare; systolic arrays)	MIMD (modern multi-core CPUs)

Speed-up Formula:

```
Speed-up(n, p) = Serial_Time(n) / Parallel_Time(n, p)
```

- n = input size, p = number of processors
- Ideal (linear): Speed-up = p
- Expected: 0 < Speed-up ≤ p

---

2. MapReduce & Hadoop (Weeks 2-3)

2.1 Work Distribution Challenges

Moving to parallel processing introduces: - **Load Balancing** — ensure all machines have equal work - **Communication Overheads** — additional stages, data transfer between tasks - **Orchestration** — master services assign tasks to machine slots - **Fault Tolerance** — machine failures, master failures, network failures - **Data Splitting** — divide data into n chunks ≥ number of pipelines

2.2 MapReduce Paradigm

Invented by Google (2004). Published as a paper, not open-source.

Core Operations:

```
Map(Key1, Value1)           → List[<Key2, Value2>]
Reduce(Key2, List[Value2])  → List<Value2>
```

Full Pipeline:

```
Input → Split → Map → Sort(by key) → Shuffle/Copy(by key) → Reduce → Output
         ↓        ↓         ↓              ↓                    ↓
       HDFS    Buffer    Memory        Network Transfer      HDFS Write
```

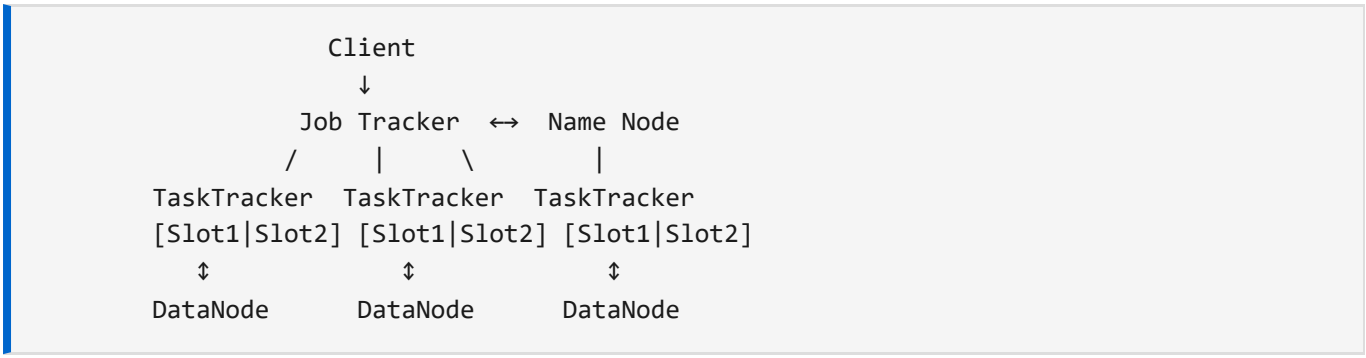
Why MapReduce was powerful: - Simple: only two operation types - Most Big Data tasks fit: Extraction = Map, Transformation = Map, Aggregation = Reduce - Clear data transfer flow

2.3 Hadoop Architecture (v1)

Yahoo!'s open-source implementation of MapReduce (2006), written in Java.

Components:

Component	Role
Client	Submits work to the cluster
Job Tracker	Receives jobs, uses NameNode to find data, assigns work to TaskTrackers
Task Tracker	Executes Map and Reduce tasks (has slots)
Name Node	Tracks location of all data on HDFS
Data Node	Stores and provides access to data blocks



Data Locality Principle: Move compute to data, not data to compute. - Scheduler priority: data-local > rack-local > remote

2.4 Hadoop Execution Flow (23 Steps)

1. Client **runs job**
2. Client gets **Job ID** from Job Tracker
3. Client **copies job resources** (Job.jar) to HDFS on all nodes
4. Client **submits job** to Job Tracker
5. Job Tracker **initializes job**
6. Job Tracker **retrieves input splits** (from NameNode)
7. Job Tracker **registers tasks** with TaskTrackers (via heartbeats)
8. TaskTracker **gets job resources** (Job.jar from local HDFS)
9. TaskTracker **launches JVM** for task
10. **Map task runs** — reads data using InputFormat
11. Map reads data from local DataNode
12. Map performs **intermediate sort** (by key, in memory)
13. Map **writes output** (using OutputFormat) to local disk
14. TaskTracker **reports map done** to Job Tracker
15. Job Tracker **retrieves map partitions** locations 16-18. Reduce TaskTracker registered, gets resources, launches JVM
16. **Reduce task runs**
17. Reduce **reads map output** (copies by key across network — "shuffle")
18. Reduce **writes final output** to HDFS
19. TaskTracker **reports reduce done**
20. Job Tracker **reports job complete** to client

2.5 Input and Output Formats

InputFormat — converts HDFS blocks to usable data: - `TextInputFormat` — line-oriented text files - `KeyValueTextInputFormat` — key-value pairs per line (tab-separated) - `SequenceFileInputFormat` — binary key-value files - `CombineFileInputFormat` — merges multiple small files - `NLineInputFormat` — split every N lines - `DBInputFormat` — reads from SQL via JDBC

OutputFormat — writes partitions to persistent storage: - `TextOutputFormat`, `SequenceFileOutputFormat`, `MapFileOutputFormat`

2.6 Hadoop Additional Concepts

Combiners: - Mini-reducers that run locally on map output before shuffle - Reduce network transfer by pre-aggregating - Must have same input/output types as the reducer - Example: In word count, combine local counts before sending to reducer

Partitioners: - Control which reducer receives which keys - Default: `HashPartitioner` — hash(key) mod numReducers - Custom partitioners for better load balancing

Distributed Cache: - Alternative to HDFS for sharing small files (jars, configs, archives) - Read-only, copied once per job to each TaskTracker - Can be private or public

Counters: - Track task progress numerically - Built-in counters for HDFS I/O, map/reduce progress - Support user-defined counters

2.7 Issues with Hadoop

Issue	Description
Small files problem	HDFS blocks are 128MB; many small files overload NameNode index
Speed	Writes to disk after every task; no caching; designed when RAM was expensive
Chaining cost	Multiple MapReduce pairs require disk write + re-read between each
Limited use-cases	No state sharing across tasks; no streaming; limited side-effects
Ease of use	Hand-written tasks in verbose Java; no pre-built abstractions

Solutions to small files: merge files, use HAR archives, use HBase for binary data

2.8 Hadoop Compression & Failure Types

Compression (recommended for all Hadoop I/O): - More data per HDFS block → lower storage/network overhead, higher throughput - Additional CPU cost for decompression (usually worth it) - Splittable formats: **BZip2, LZO, Snappy** (important for MapReduce input splits) - Non-splittable: GZip, ZLib (entire file must go to one mapper)

Task Failure Detection: - Runtime exception → error reported to TaskTracker by child JVM - Child JVM dies → TaskTracker informed by OS - Task hung → detected via heartbeat timeouts - Job Tracker reschedules to different TaskTracker; multiple attempts per task

TaskTracker Failure: - Detected by Job Tracker via missing heartbeats or multiple task failures - Removed from pool; tasks rescheduled to other TaskTrackers - Failed TaskTrackers reconsidered after timeout

2.9 Hadoop Versioning

Version	Key Features
V1	Basic MapReduce + HDFS
V2	YARN (Yet Another Resource Negotiator) — better scheduling
V3	Multiple NameNode support, erasure coding, container/GPU support

Exam Template — "Design a MapReduce job for X": 1. Define the **input format** (what does each record look like?) 2. Define the **Map function**: what key-value pairs does it emit? 3. Define the **Sort/Shuffle**: how are keys grouped? 4. Define the **Reduce function**: what aggregation is performed? 5. Define the **output format**

3. Apache Spark (Weeks 3-4)

3.1 Origin and Motivation

- Started as UC Berkeley class project (2009), open-sourced 2010
- Designed to fix Hadoop's shortcomings
- **Key motivation**: By 2009, RAM costs had dropped 20x — in-memory processing became feasible
- Core idea: **MapReduce+ in Memory** — eliminate disk read/write between stages

3.2 Spark vs Hadoop Comparison

Feature	Hadoop	Spark
Storage	Disk between stages	**In-memory** (RAM)
Speed	Slower (disk I/O)	**10-100x faster**
Chaining	Expensive (disk writes)	Cheap (memory transfers)
API	Low-level Java	**High-level (Java, Scala, Python, R)**
Processing	Batch only	**Batch + Streaming**
Caching	None	**In-memory caching**
Fault tolerance	Disk replication	**RDD lineage** (recompute from DAG)
Ease of use	Verbose	**Concise, lambda expressions**

3.3 Resilient Distributed Datasets (RDDs)

RDD = immutable, distributed collection of records (like a table).

Key properties: - **Resilient**: Can be reconstructed if lost (via lineage/DAG) - **Distributed**: Partitioned across cluster nodes - **Dataset**: Collection of records with optional schema

RDDs support two types of operations: - **Transformations**: Create new RDD from existing (lazy — not executed immediately) - **Actions**: Trigger computation and return result to driver or storage

3.4 Lazy Evaluation — Why It Matters

Spark does NOT execute transformations immediately. Instead: 1. Builds a **DAG** (Directed Acyclic Graph) of transformations 2. Only executes when an **action** is called 3. Spark optimises the DAG before execution (e.g., pipelining, predicate pushdown)

Benefits: - Avoids unnecessary computation - Enables optimisation of the full pipeline - Reduces intermediate data materialization

3.5 DAGs and Stage Creation

DAG = Directed Acyclic Graph representing the computation plan.

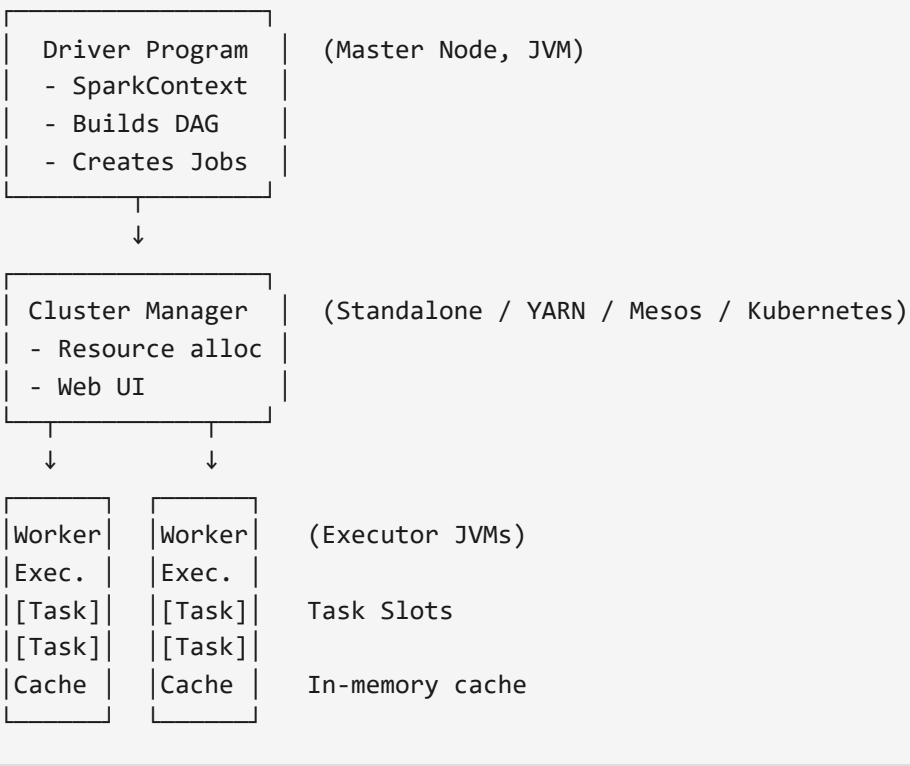
Narrow vs Wide Dependencies:

Type	Description	Example	Shuffle?
Narrow	Each parent partition used by at most one child partition	map, filter, flatMap	No
Wide	Each parent partition used by multiple child partitions	groupByKey, reduceByKey, join	**Yes**

Stage boundaries: Every wide dependency creates a new stage (requires shuffle).

```
Stage 1: read → map → filter     (narrow dependencies, pipelined)
                    ↓ shuffle
Stage 2: reduceByKey → map        (narrow dependencies, pipelined)
                    ↓ action
Result returned to driver
```

3.6 Spark Architecture



Execution Flow: 1. Driver builds a **Job Jar** with serialized functions 2. Job Jar sent to Cluster Manager 3. Cluster Manager allocates work to Executors 4. Executors read data partitions and run tasks 5. Intermediate results stored in executor **memory cache** 6. Spark co-locates tasks to machines where data is cached 7. When all tasks complete, action result sent to driver

Key terminology: - **Job**: Parallel computation triggered by an action - **Stage**: Set of tasks with no shuffle boundary between them - **Task**: Single unit of work on one partition

Spark Execution Phases (7 Steps): 1. Client creates **SparkContext** → initialises Driver, registers with Cluster Manager 2. Client creates **input RDDs** (partitioned across cluster) 3. Client defines **transformations** → builds DAG (lazy, no computation yet) 4. Client calls an **action** → triggers DAG submission 5. **DAGScheduler** builds stages of tasks, groups by dependencies, submits to TaskScheduler 6. **TaskScheduler** ships tasks to Executors (serialised RDD lineage + transformations), monitors progress, handles stragglers via speculative execution 7. Results returned to Driver; executors shut down when job completes

Important Differences from Hadoop: - No defined persistent storage layer (Spark uses temporary storage if RAM exhausted) - Driver must remain alive until job finishes - Each job gets its own Executor (no memory sharing between jobs → faster startup than Hadoop)

3.7 Transformations

Basic Transformations (Narrow):

Transformation	Signature	Description
`map`	`T → U`	Apply function to each element
`filter`	`T → Boolean`	Keep elements where function returns true
`flatMap`	`T → Iterator<U>`	Map then flatten (one-to-many)
`mapToPair`	`T → Tuple2<K,V>`	Convert to key-value pairs

Advanced Transformations (Wide — cause shuffle):

Transformation	Description	Notes
`groupByKey`	Group values by key	Expensive! Moves all data
`reduceByKey`	Combine values by key using function	**Preferred** over groupByKey (local pre-aggregation)
`join`	Join two datasets by key	Wide dependency
`cogroup`	Group two datasets by key	Returns iterables per key
`sortByKey`	Sort by key	Wide dependency

> **Exam Tip**: Always prefer `reduceByKey` over `groupByKey` — it reduces network transfer by combining locally first (like a combiner in Hadoop).

3.8 Actions

Action	Description
`collect()` / `collectAsList()`	Return all elements to driver
`count()`	Count number of elements
`reduce()`	Aggregate using function
`first()`	Return first element
`take(n)`	Return first n elements
`saveAsTextFile()`	Write to text file
`foreach()`	Apply function to each element (no return)

3.9 Partitioning

- Initial partitioning set when data is loaded (one partition per file by default)
- Single-file readers create only one partition (bad for parallelism!)
- **Repartition**: `dataset.repartition(n)` — expensive but enables parallelism
- **Recommended**: 2-3x the number of task slots
- Default: `HashPartitioner` — hash(key) mod numPartitions
- Custom partitioners possible (requires RDD level, not Dataset)

3.10 Broadcast Variables & Accumulators

Broadcast Variables: - Share read-only data efficiently across all executors - One copy per executor (not per task) - Created via: `javaSparkContext.broadcast(data)` - Accessed in functions via: `broadcastVar.value()` - Use case: lookup tables, configuration maps

Accumulators: - Global counters updated by tasks, aggregated at driver - Types: `LongAccumulator`, `DoubleAccumulator`, collection accumulator - Created via: `spark.sparkContext().longAccumulator()` - Updated in tasks, read in driver after an action completes - `value()` is NOT an action — ensure an action runs before reading

3.11 Advantages and Disadvantages of Spark

Advantages: - Much faster than Hadoop (in-memory processing) - Supports batch AND streaming (micro-batches) - Rich API with many built-in transformations - Lazy evaluation enables optimisation - Fault tolerant via RDD lineage - Multi-language support (Java, Scala, Python, R)

Disadvantages: - Higher memory requirements (RAM is expensive at scale) - Not suitable for all workloads (e.g., very large datasets that don't fit in memory) - Micro-batch streaming has higher latency than true stream processing - Complex debugging (lazy evaluation makes stack traces confusing)

3.12 Batch vs Stream Processing

Aspect	Batch	Stream (Spark Streaming)
Data	Static/complete dataset	Continuous data flow
Processing	All at once	**Micro-batches** (small time windows)
Latency	High (minutes to hours)	Low (seconds)
Framework	Spark Core	Spark Streaming / Structured Streaming
Use case	ETL, analytics, ML training	Real-time dashboards, fraud detection

Spark Streaming: Treats stream as sequence of small batches (micro-batches). Not true real-time — introduces small delay. Alternative: Apache Flink for true event-at-a-time processing.

3.13 Streaming Challenges — State & Back-Pressure

Stateful Applications: - A task that has persistent state changing over time based on input - Problem: if data is randomly distributed across parallel task instances, state becomes inconsistent - Solution: **KeyBy** — route all items with the same key to the same task instance - Shared state (state needed across task instances) is even harder — synchronisation adds latency, failure can corrupt global state

Back-Pressure: - When downstream tasks can't keep up with upstream tasks - Buffers fill up → previous tasks can't output → their buffers fill → cascading failure - Eventually the system fails when new items arrive with nowhere to go - **Avoid by**: proper capacity planning, load shedding, or adaptive rate control

State Management Warning: "State management is a major problem for both streaming and batch-based operations. Try to avoid including state in transformations where possible!"

> **Model Answer — "Why Spark over Hadoop?"** 1. **In-memory processing** — avoids costly disk I/O between stages [2 marks] 2. **DAG execution engine** — optimises full pipeline, not just individual map/reduce steps [2 marks] 3. **Rich API** — transformations beyond map/reduce; lambda expressions reduce code [1 mark] 4. **Unified engine** — supports batch AND streaming on same platform [1 mark] 5. **Fault tolerance via lineage** — recompute lost partitions from DAG rather than replication [1 mark]

---

4. Distributed Storage & HDFS (Weeks 5-6)

4.1 Persistent Storage Primer

Hard Disk Drive (HDD): - Magnetic platters, 7200 RPM - Slow random reads, medium sequential reads, slow writes - Cheap, ~4-6 year lifespan - Seek time dominates performance

Solid State Drive (SSD): - NAND flash memory, no moving parts - Fast random reads, fast sequential reads, medium writes - More expensive, ~5-10 year lifespan - Write endurance is limited (cells wear out)

Host Interfaces (speed): - Fibre Channel (128 Gbit/s) — servers only - PCIe 4.0 x4 (63 Gbit/s) — M.2 NVMe SSDs - PCIe 3.0 x4 (31.5 Gbit/s) - SAS-3 (12 Gbit/s) — servers - SATA 3.0 (6 Gbit/s) - USB 3.0 (10 Gbit/s)

Key insight: Even fast storage can be bottlenecked by the host interface.

4.2 Distributed vs Decentralized Storage

Type	Description
Distributed	Data spread across multiple machines, managed by central coordinator
Decentralized	No single coordinator; peer-to-peer management

4.3 NFS (Network File System)

- Developed in 1984 by Sun Microsystems
- Client-server model: NFS server exports directories, clients mount them
- **Transparent access**: remote files look like local files

NFS Architecture:

```
Client 1 ──→ NFS Server ──→ Storage
Client 2 ──→     ↑
Client 3 ──→  (single point of failure)
```

NFS Limitations: - **Single point of failure** — server goes down, all access lost - **Scalability** — limited by single server's I/O bandwidth - **Network bottleneck** — all traffic funnels through one server - **No built-in replication** — must use RAID at server level

4.4 HDFS Architecture

Hadoop Distributed File System — designed for large files, streaming reads, commodity hardware.

Design Principles: - Store very large files (GB to TB) - Streaming data access (write once, read many) - Run on commodity hardware (expect failures) - Optimised for **throughput** over latency

Components:

Component	Role	Count
NameNode	Metadata master — stores file→block mapping, block→DataNode mapping	1 (+ secondary/standby)
DataNode	Stores actual data blocks, sends heartbeats to NameNode	Many (100s-1000s)

```
        NameNode
       (metadata)
      /    |     \
DataNode DataNode DataNode
[blocks] [blocks] [blocks]
```

4.5 HDFS Block Size

- Default: **128MB** (was 64MB in older versions)
- Much larger than typical OS block size (4KB)
- **Rationale**: Minimise disk seek overhead
- Seek time ~10ms, transfer rate ~100MB/s
- To make seek time 1% of transfer: block_size = 100MB/s × 10ms × 100 = 100MB
- Rounded up to 128MB

Benefits of large blocks: - Fewer blocks = smaller NameNode metadata - Fewer seeks = better throughput - Reduces number of network connections

Drawback: Wastes space for small files (each file ≥ 1 block in metadata)

4.6 HDFS Write Pipeline (Step-by-Step)

1. Client calls `create()` on NameNode to write a new file
2. NameNode creates a **block ID** for first block, decides which DataNodes will host replicas
3. Client receives:
4. A **lease** (like a lock — 1 writer, many readers)
5. Block ID and DataNode locations
6. Client creates a **pipeline** of DataNodes based on proximity (minimise total network distance)
7. Client pushes data in **packets** (64KB) into a local buffer
8. When buffer full, packets sent through the pipeline:
9. DN1 receives → forwards to DN2 → forwards to DN3
10. Acknowledgements flow back: DN3 → DN2 → DN1 → Client
11. Repeat for each block of the file
12. Packets sent **asynchronously** (next sent before previous ACKed)
13. `hflush()` ensures all packets written before it are visible to readers

4.7 HDFS Read Pipeline (Step-by-Step)

1. Client contacts NameNode with file pathname → NN returns block IDs + DataNode locations (sorted by network distance)

2. Client contacts the **closest DataNode** for each block directly
3. DataNode streams block data in **packets** (64KB) along with checksum
4. Client **verifies checksum** — if verification fails, reports to NameNode and tries next DataNode
5. If DataNode fails or is unreachable, client marks it as faulty and tries the next replica
6. Client caches DataNode locations to avoid repeated NameNode queries

> **Exam Key Point** (from 2018 marking scheme): The read failure/recovery part is worth ~40% of read protocol marks. Always mention: checksum verification, fallback to next DataNode, and reporting failures to NameNode.

4.7b NameNode Recovery (Frequently Tested)

When NameNode restarts after failure: 1. Reads **checkpoint file** (snapshot of filesystem state) from local disk 2. Replays **journal file** (write-ahead log of mutations since last checkpoint) 3. This restores directory structure and block→file mappings 4. Waits for **DataNodes to register** and send block reports (lists of block IDs + sizes) 5. Stores block→DataNode mappings in memory 6. System enters **safe mode** until sufficient replicas are confirmed

4.8 HDFS Replica Placement Policy

Default replication factor: **3 copies**

```
Replica 1: Same node as writer (or random node if writer is external)
Replica 2: Different rack from Replica 1 (remote rack)
Replica 3: Same rack as Replica 2, different node
```

Rationale: - Replica 1 on writer's node: fast local write - Replica 2 on different rack: survives rack failure - Replica 3 on same rack as R2: reduces inter-rack bandwidth (only 1 cross-rack transfer needed)

4.9 HDFS Block States and Recovery

Block States:

State	Meaning
RBW (Replica Being Written)	Block is currently being written
RWR (Replica Waiting to be Recovered)	Writer failed; block needs recovery
RUR (Replica Under Recovery)	Recovery in progress
Finalized	Block is complete and sealed

Lease Recovery: - If client dies mid-write, its lease expires - NameNode triggers lease recovery - Selects a DataNode as recovery leader - Leader coordinates to finalize all replicas to a consistent state

Pipeline Recovery: - If a DataNode in the write pipeline fails: - Client removes failed node from pipeline - Continues writing with remaining nodes - NameNode schedules re-replication to restore target count

4.10 DataNode ↔ NameNode Communication

Heartbeats: - DataNodes send heartbeats to NameNode every **3 seconds** - If NameNode misses heartbeats for **10 minutes**, marks DataNode as dead - Heartbeats carry: available storage, number of transfers in progress

Block Reports: - DataNodes send full block reports periodically - Lists all blocks stored on that DataNode - NameNode uses this to maintain its block→DataNode mapping

4.11 Re-replication and Cluster Balancing

Re-replication triggers: - DataNode failure (detected via missed heartbeats) - Replica corruption (detected via checksum verification) - Replication factor increase - DataNode disk failure

Cluster Balancer: - Redistributes blocks to balance storage across DataNodes - Runs as a background process - Moves blocks from over-utilized to under-utilized nodes - Respects rack placement policy

4.12 RAID Comparison

Level	Method	Min Disks	Space Efficiency	Fault Tolerance
RAID 0	Striping	2	100%	None
RAID 1	Mirroring	2	50%	1 disk failure
RAID 5	Striping + distributed parity	3	(n-1)/n	1 disk failure
RAID 6	Striping + double parity	4	(n-2)/n	2 disk failures
RAID 10	Striped mirrors	4	50%	1 per mirror pair

Model Answer — "NFS vs HDFS":

Aspect	NFS	HDFS
Architecture	Centralised server	Distributed (NameNode + DataNodes)
Failure	Single point of failure	Tolerant (replication, re-replication)
Scalability	Limited by single server	Horizontally scalable
File size	Any	Optimised for large files (>128MB)
Access pattern	Random read/write	Write once, read many (streaming)
Replication	RAID at server level	Built-in (3x replication across racks)
CAP	CA (no partition tolerance)	CP (consistent, partition tolerant)
Latency	Low for small files	High for small files, good throughput for large

5. CAP Theorem & BASE (Week 6)

5.1 CAP Theorem

Statement: In a distributed system, you can only guarantee **two** of three properties simultaneously:

Property	Description
Consistency (C)	Every read receives the most recent write (all nodes see the same data at the same time)
Availability (A)	Every request receives a response (system is always operational)
Partition Tolerance (P)	System continues operating despite network partitions between nodes

Why only 2?: In a network partition, you must choose: - Respond with potentially stale data → sacrifice **Consistency** (keep A+P) - Wait for partition to heal → sacrifice **Availability** (keep C+P) - You cannot avoid partitions in distributed systems → P is usually mandatory

5.2 CAP Classifications

Category	Properties	Examples	Trade-off
CA	Consistency + Availability	Traditional RDBMS, NFS	No partition tolerance (single node)
CP	Consistency + Partition Tolerance	**HBase**, MongoDB, ZooKeeper, **HDFS**	May be unavailable during partition
AP	Availability + Partition Tolerance	**Cassandra**, DynamoDB, CouchDB	May return stale data

NFS under CAP: CA — single server, no partition tolerance (goes down = unavailable) **HDFS under CAP**: CP — consistent reads (NameNode is authoritative), partition tolerant (replication), but may be unavailable if NameNode fails

5.3 ACID vs BASE

ACID (Traditional DB)	BASE (NoSQL/Distributed)
Atomicity — all or nothing	**B**asically **A**vailable — system always responds
Consistency — valid state transitions	**S**oft State — state may change without input (propagation)
Isolation — concurrent transactions don't interfere	**E**ventually Consistent — given time, all nodes converge
Durability — committed = permanent	

When to use which: - ACID: Banking, critical transactions where correctness is paramount - BASE: Social media, analytics, systems where eventual consistency is acceptable

> **Model Answer — "Classify system X under CAP":** 1. Identify if the system is distributed (if not → CA) 2. State which two properties it prioritises 3. Explain what is sacrificed and the practical impact 4. Give a concrete example of the trade-off in action

---

6. LSM Trees & Bloom Filters (Week 6)

6.1 The Index Problem

In distributed systems, we need efficient ways to find data. Traditional B-Tree indexes are optimised for reads but expensive for writes (random I/O). We need write-optimised structures.

6.2 Log-Based Indexing

Approach: Append all writes to a sequential log file.

Advantage	Disadvantage
Very fast writes (sequential I/O)	Slow reads (must scan entire log)
Simple implementation	Log grows unboundedly
Natural write ordering	No efficient point lookups

6.3 LSM Tree (Log-Structured Merge Tree)

Components:

```
Write → WAL (Write-Ahead Log) → MemTable (in-memory, sorted)
                                    ↓ (when full)
                                SSTable (on-disk, sorted, immutable)
                                    ↓
                                Compaction (merge SSTables)
```

Component	Description
WAL (Write-Ahead Log)	Durable log for crash recovery; appended before MemTable
MemTable	In-memory sorted data structure (e.g., red-black tree)
SSTable (Sorted String Table)	On-disk sorted, immutable file; indexed for fast lookups
Index	Sparse index into SSTables for efficient point lookups
Compactor	Background process that merges SSTables

6.4 LSM Write Process (4 Steps)

1. Write entry to **WAL** (for crash recovery)
2. Insert into **MemTable** (sorted, in-memory)
3. When MemTable reaches threshold → flush to disk as new **SSTable**
4. Background **compaction** merges SSTables periodically

6.5 LSM Read Process (3 Steps)

1. Check **MemTable** first (most recent writes)
2. Check **SSTables** from newest to oldest (using sparse index)
3. If not found in any → key does not exist

6.6 Compaction and Tombstones

- **Compaction**: Merges multiple SSTables into one, removing duplicates (keeping newest)
- **Tombstones**: Special markers for deleted keys (can't just remove from immutable SSTable)
- Tombstones are removed during compaction when no older SSTable references the key

6.7 Bloom Filters

Purpose: Probabilistic data structure for fast set membership testing. - "Is this key possibly in this SSTable?" → **Yes** (maybe) or **No** (definitely not) - **No false negatives** — if it says "no", the key is definitely not there - **Possible false positives** — may say "yes" when key isn't there

Construction: - Bit array of **m** bits (all initially 0) - **k** independent hash functions - To add element: compute k hash values, set those bit positions to 1 - To query: compute k hash values, check if ALL positions are 1 - All 1s → **possibly present** (could be false positive) - Any 0 → **definitely not present**

6.8 Bloom Filter Formulas

False positive probability:

```
p_fp = (1 - e^(-kn/m))^k
```

Where: n = number of elements, m = number of bits, k = number of hash functions

Optimal number of hash functions:

```
k_opt = (m/n) × ln(2) ≈ 0.693 × (m/n)
```

Required bits for target false positive rate:

```
m = -(n × ln(p_fp)) / (ln(2))^2
```

6.9 Worked Example

Given: n = 1000 elements, target p_fp = 5% (0.05)

Step 1 — Calculate m:

```
m = -(1000 × ln(0.05)) / (ln(2))^2
m = -(1000 × (-2.996)) / (0.480)
m = 2996 / 0.480
m ≈ 6,235 bits (≈ 780 bytes)
```

Step 2 — Calculate optimal k:

```
k = (6235/1000) × ln(2) = 6.235 × 0.693 ≈ 4.32 → round to 4 or 5
```

Result: ~6,235 bits and 4-5 hash functions for 5% false positive rate with 1000 elements.

> **Exam Tip**: Bloom Filters are used in LSM Trees to avoid unnecessary disk reads. Before reading an SSTable, check the Bloom Filter — if it says "no", skip the SSTable entirely.

---

7. BigTable & HBase (Week 7)

7.1 BigTable — Google's Distributed Database

- Developed by Google (2005) after MapReduce and GFS
- Designed to manage Google's rapidly growing index (20x growth 2000-2004)
- A **sparse, distributed, persistent multi-dimensional sorted map**

7.2 BigTable Data Model

```
(row_key, column_family:qualifier, timestamp) → value
```

Concept	Description
Row Key	Unique identifier, sorted lexicographically; determines distribution
Column Family	Group of related columns; defined at table creation; unit of access control
Column Qualifier	Specific column within a family; created dynamically
Timestamp	Version control; multiple versions of same cell stored
Cell	Intersection of row × column × timestamp

Example — Web table:

```
Row Key: "com.google.www" (reversed URL for locality)
Column Family: "contents" → full HTML
Column Family: "anchor"   → anchor:cnn.com = "CNN", anchor:bbc.com = "BBC"
Timestamps: t1, t2, t3 (multiple versions)
```

Key design decisions: - Rows sorted by key → related data stored together - Column families are few and fixed; qualifiers are many and dynamic - Timestamps enable versioning without separate version tables

7.3 Tablets/Regions

- Table split into **tablets** (called **regions** in HBase)
- Each tablet: a contiguous range of row keys
- Distributed across tablet servers
- When a tablet grows too large → **split** into two tablets
- Tablets are the unit of distribution and load balancing

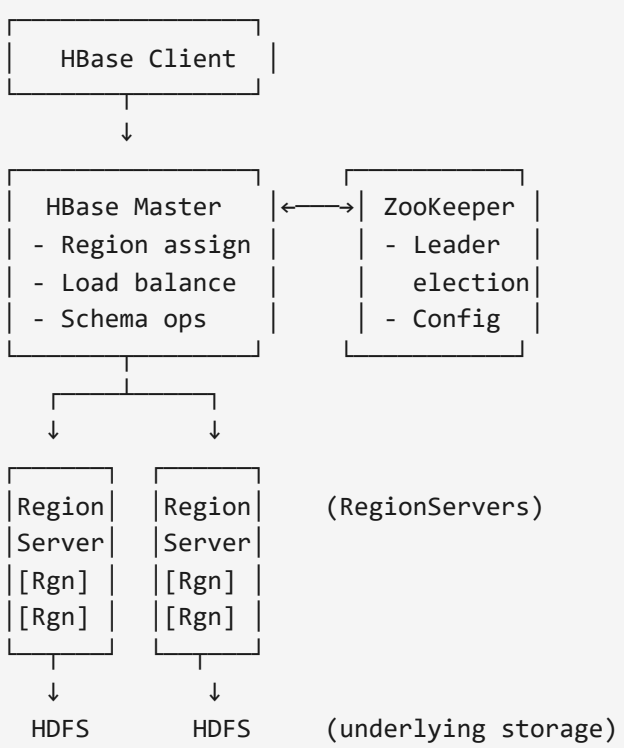
7.4 Region Splitting

- Automatic when region exceeds threshold size
- Split point chosen at midpoint of row key range

- Parent region goes offline briefly during split
- Two child regions assigned to (potentially different) RegionServers

7.5 HBase Architecture

Open-source implementation of BigTable (released 2008 by Powerset/Microsoft).



Components:

Component	Role
HBase Master	Assigns regions to RegionServers, handles schema changes, load balancing
RegionServer	Hosts regions, handles reads/writes for its regions
ZooKeeper	Coordination service — leader election, configuration, health monitoring
HDFS	Underlying persistent storage for HBase data

7.6 HBase Storage — Uses LSM Trees!

Each RegionServer uses an LSM Tree structure: 1. **HLog (WAL)** — write-ahead log for durability 2. **MemStore** — in-memory sorted buffer (one per column family) 3. **HFile (StoreFile)** — on-disk SSTable equivalent 4. **Compaction** — merges HFiles periodically

Write path: Client → HLog → MemStore → (flush) → HFile on HDFS Read path: MemStore → Block Cache → HFiles (use Bloom Filters!)

7.7 Column Store vs Row Store

Aspect	Row Store	Column Store
Storage	All columns of a row together	Each column stored separately
Read pattern	Good for full-row reads	Good for aggregation queries on few columns
Write pattern	Fast row inserts	Slower (multiple column files)
Compression	Lower (mixed data types)	Higher (same data type per column)
Example	Traditional RDBMS	BigTable/HBase, Cassandra

BigTable/HBase is a **column-family store** — a hybrid. Data within a column family is stored together, but different families can be on different storage.

7.8 HBase Data Lookup Procedure (Tested 2016, 2019)

When a client requests a data item: 1. Client contacts **ZooKeeper** to get the RegionServer hosting the **META table** 2. Client queries META table to find which RegionServer hosts the region containing the target row key 3. Client contacts that **RegionServer** directly for read/write 4. RegionServer mediates between client and HDFS — reads data from DataNodes, caches in block cache 5. Client **caches** the region→RegionServer mapping for future requests (avoids repeated META lookups)

> **Key exam point**: The HBase Master is NOT part of the client-server data path! This is how HBase avoids scalability issues with the master.

7.9 HBase On-Disk Storage Format (Tested 2018)

- Data stored as **key-value pairs** (not traditional rows)
- Key = composite of: rowkey + column family + column qualifier + timestamp (+ sizes)
- Value = column value (+ size in bytes)
- Key-value pairs **sorted by composite key** → all cells for same rowkey are contiguous
- This is different from traditional RDBMS row storage

7.10 Why Not RDBMS for Big Data? (Tested 2016)

1. **Normalisation problem**: RDBMS uses many tables with foreign keys → accessing data across tables requires expensive multi-way JOINs, intolerable at scale
2. **Consistency overhead**: Strong ACID semantics require expensive distributed concurrency control and commit protocols, often unnecessary for Big Data applications

---

8. ZooKeeper & DHT (Week 7)

8.1 ZooKeeper — Distributed Coordination Service

Purpose: Provide reliable, ordered coordination primitives for distributed systems.

Used by: HBase, Kafka, Hadoop (HDFS HA), Cassandra, Solr, and many others.

Key Services: - **Leader Election** — choose one node as master among candidates - **Configuration Management** — distribute config to all nodes - **Group Membership** — track which nodes are alive - **Distributed Locks** — coordinate exclusive access - **Barrier Synchronization** — synchronise phases of computation

8.2 ZooKeeper Data Model — ZNodes

- Hierarchical namespace (like a filesystem): `/app/config/database`
- Each ZNode can store a small amount of data (~1MB max)

ZNode Types:

Type	Description
Persistent	Exists until explicitly deleted
Ephemeral	Automatically deleted when creating client's session ends
Sequential	Has a monotonically increasing counter appended to name

Watches: - Clients can set watches on ZNodes - Notified when ZNode changes (data change, deletion, children change) - One-time triggers — must re-register after each notification

Leader Election Example: 1. Each candidate creates an ephemeral sequential ZNode under `/election/` 2. Node with lowest sequence number becomes leader 3. If leader dies → its ephemeral node deleted → next in line becomes leader

8.3 Distributed Hash Tables (DHT)

Purpose: Distribute key-value data across nodes without a central coordinator.

Consistent Hashing: - Nodes and keys mapped to positions on a hash ring (0 to 2^n - 1) - Each key assigned to the next node clockwise on the ring - When nodes join/leave, only nearby keys need to move

Benefits: - Minimal key redistribution when nodes change - Decentralised — no single point of failure - Scalable — add nodes without full rehash

8.4 Overlay Networks

- Logical network built on top of physical network
- Each node knows a subset of other nodes (routing table)
- Messages routed through overlay to reach target node
- Examples: Chord, Pastry, CAN

8.5 Order-Preserving Hash Functions

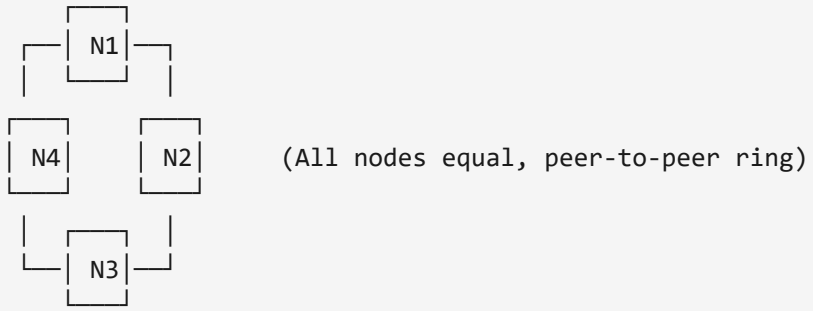
- Regular hash functions scatter related keys across the ring
- Order-preserving functions keep keys with similar values close together
- Enables **range queries** on DHTs
- Trade-off: potential for hotspots (popular key ranges on same node)

---

9. Cassandra (Week 9)

9.1 Cassandra Architecture

- Originally developed at Facebook, open-sourced 2008
- **Peer-to-peer** architecture — no master node (unlike HBase)
- All nodes are equal — any node can serve any request
- Based on Amazon's Dynamo (architecture) + Google's BigTable (data model)



9.2 Consistent Hashing with Virtual Nodes (vnodes)

- Each physical node owns multiple **virtual nodes** (vnodes) on the hash ring
- Default: 256 vnodes per physical node
- Benefits:
- Better load distribution
- Faster rebalancing when nodes join/leave
- Heterogeneous hardware (more powerful nodes get more vnodes)

9.3 Replication Strategy

Strategy	Description
SimpleStrategy	Place replicas on next N nodes clockwise on ring
NetworkTopologyStrategy	Specify replicas per data centre; uses rack-awareness

Replication Factor (N): Number of copies of each piece of data (typically 3).

9.4 Consistency Levels

Level	Description	Use Case
ONE	Ack from 1 replica	Fastest, lowest consistency
QUORUM	Ack from ⌊N/2⌋+1 replicas	Good balance
ALL	Ack from all N replicas	Strongest consistency, lowest availability
LOCAL_QUORUM	Quorum within local data centre	Multi-DC deployments

9.5 Quorum-Based Replication

Formula for strong consistency:

```
R + W > N
```

Where: R = read consistency level, W = write consistency level, N = replication factor

Example: N=3, W=QUORUM(2), R=QUORUM(2) → 2+2=4 > 3 ✓ (strongly consistent)

Read-your-own-write consistency: Set W + R > N to guarantee a reader will see their own writes.

9.6 Cassandra vs HBase Comparison

Aspect	Cassandra	HBase
Architecture	**Peer-to-peer** (no master)	Master-slave (HBase Master)
CAP	**AP** (available, partition tolerant)	**CP** (consistent, partition tolerant)
Single point of failure	**None**	HBase Master (mitigated by HA)
Consistency	Tunable (ONE to ALL)	Strong
Write performance	**Very high** (always writable)	High
Read performance	Good	**Better for random reads**
Data model	Wide column store	Wide column store
Query language	**CQL** (Cassandra Query Language)	Java API, Thrift
Use case	High write throughput, availability critical	Strong consistency required

9.7 Cassandra Data Model

Similar to BigTable/HBase: - **Keyspace** (like a database) - **Table** (column family) - **Partition Key** — determines which node stores the data - **Clustering Key** — determines sort order within a partition - **Primary Key** = Partition Key + Clustering Key(s)

---

10. Cluster Management & Ethics (Weeks 9-10)

10.1 Cluster Management Systems

System	Description
YARN	Hadoop's resource manager (Yet Another Resource Negotiator); manages cluster resources for Hadoop ecosystem
Mesos	Apache project; two-level scheduling; supports multiple frameworks
Kubernetes (K8s)	Container orchestration; manages containerised applications; most popular today

10.2 Software Sandboxes

Containers: - Lightweight, share host OS kernel - Fast startup (seconds) - Less overhead than VMs - Examples: Docker, Podman - Isolation via Linux namespaces and cgroups

Virtual Machines: - Full OS copy per VM - Slower startup (minutes) - Stronger isolation (separate kernel) - Higher resource overhead - Examples: VMware, VirtualBox, KVM

Aspect	Container	VM
Startup	Seconds	Minutes
Overhead	Low	High
Isolation	Process-level	Hardware-level
Image size	MBs	GBs
Portability	Very portable	Less portable

10.3 OKD4/OpenShift

- Red Hat's Kubernetes distribution
- **Level 0**: Basic Kubernetes concepts (pods, services, deployments)
- **Level 1**: Operators for automated application management
- **Level 2**: Full lifecycle management with monitoring, logging, and CI/CD

10.4 Legal and Ethical Considerations

GDPR (General Data Protection Regulation): - EU regulation on data privacy and protection - Key principles: consent, purpose limitation, data minimisation, right to erasure - Applies to any organisation processing EU citizens' data

Key Ethical Issues in Big Data: - **Privacy**: Collection and use of personal data without consent - **Bias**: Algorithmic bias from biased training data - **Transparency**: Black-box algorithms making decisions about people - **Data ownership**: Who

owns the data? Who can use it? - **Surveillance**: Mass surveillance capabilities - **Re-identification**: Anonymised data can sometimes be re-identified

---

11. Exam Strategy & Model Answers

11.1 Answer Structure Template (A-Grade)

For **conceptual questions** (e.g., "explain X", "discuss Y"):

```
1. DEFINITION — State what X is in one clear sentence [1-2 marks]
2. COMPONENTS — List key parts/properties [2-3 marks]
3. HOW IT WORKS — Explain the mechanism step-by-step [3-5 marks]
4. WHY — Explain the motivation/rationale [1-2 marks]
5. TRADE-OFFS — Discuss advantages/disadvantages or comparison [2-3 marks]
```

For **scenario questions** (e.g., "Is this a Big Data problem?"):

```
1. IDENTIFY — Name the relevant concept/dimension [1 mark per concept]
2. APPLY — Explain how it applies to the specific scenario [1 mark per explanation]
3. CONCLUDE — Summary judgement with justification [1-2 marks]
```

For **design questions** (e.g., "Design a MapReduce job"):

```
1. INPUT — Define input format and key-value structure [2 marks]
2. MAP — Define map function with example input→output [3-4 marks]
3. SHUFFLE — Explain how data is grouped [1-2 marks]
4. REDUCE — Define reduce function with example [3-4 marks]
5. OUTPUT — Define output format [1 mark]
```

11.2 Top 10 Most-Asked Questions (from 2015-2023 papers)

1. HDFS Read/Write Protocols (asked in 2015, 2017, 2018, 2019, 2020, 2023) - Know the step-by-step process for both (see Section 4.6 and 4.7) - Include: NameNode interaction, pipeline construction, packet buffering, ACKs

2. "Is this a Big Data problem?" — Identify dimensions (2018, 2019, 2020, 2023 Practice, 2023) - Apply the V's to the given scenario - Always explain WHY each dimension applies, not just name it

3. MapReduce Job Design (2015, 2016, 2017, 2018, 2020) - Follow the template: Input → Map → Sort → Reduce → Output - Include concrete examples with sample data

4. NFS vs HDFS Comparison (2015, 2017, 2019, 2023) - Use the comparison table in Section 4.12 - Always mention: scalability, fault tolerance, single point of failure, file size optimisation

5. Spark vs Hadoop Comparison (2018, 2019, 2020, 2023) - Use table in Section 3.2 - Key points: in-memory vs disk, DAG vs linear, API richness, streaming support

6. CAP Theorem Classification (2017, 2018, 2019, 2023) - Define C, A, P first - Classify the given system - Explain what is sacrificed

7. Bloom Filter Calculation (2018, 2020, 2023) - Know the formulas (Section 6.8) - Be able to calculate m, k, and p_fp - Explain what false positives mean in context

8. BigTable/HBase Data Model (2016, 2017, 2019, 2023) - Cell identification: (row_key, column_family:qualifier, timestamp) - Explain column families vs qualifiers - Region splitting

9. LSM Tree Write/Read Process (2019, 2020, 2023) - Write: WAL → MemTable → SSTable → Compaction - Read: MemTable → SSTables (newest first) - Tombstones for deletes

10. Cassandra Consistency Levels / Quorum (2019, 2020, 2023) - R + W > N formula - Explain trade-off between consistency and availability - Compare with HBase (strong vs tunable consistency)

11.3 Common "Trick" Areas and Pitfalls

1. **Don't confuse V's**: Velocity (speed of arrival) ≠ Variability (changing patterns). Volume is obvious but not always sufficient alone.
2. **Combiner ≠ Reducer**: Combiners run locally on map output, must have same input/output types, and are optional. Reducers are mandatory final aggregation.
3. **reduceByKey vs groupByKey**: Always say reduceByKey is preferred (local pre-aggregation reduces shuffle).
4. **HDFS block size rationale**: It's about minimising seek time as a percentage of transfer time, NOT about matching file sizes.
5. **RDD lineage for fault tolerance**: Spark doesn't replicate data; it recomputes lost partitions from the DAG. This is cheaper than replication for in-memory data.
6. **CAP — Partition tolerance is mandatory**: In any distributed system, network partitions WILL happen. The real choice is C vs A during partitions.
7. **Bloom Filters — no false negatives**: If the filter says "not present", it's DEFINITELY not present. False positives are acceptable (just do an extra disk read).
8. **HBase uses HDFS**: HBase stores its data files (HFiles) on HDFS. It's a database layer ON TOP of HDFS, not a replacement.
9. **Cassandra is AP, HBase is CP**: This is a frequent exam question. Cassandra sacrifices consistency for availability; HBase does the opposite.
10. **LSM Trees are write-optimised**: Sequential writes (append-only). Reads may need to check multiple SSTables (mitigated by Bloom Filters and compaction).

11.4 Time Management

For a 75-mark, 1.5-hour exam with 3 questions: - **25 marks per question → 30 minutes per question** - **~1.2 minutes per mark** - Spend first 2 minutes reading the full paper - Don't spend more than 35 minutes on any one question — move on and come back

Mark allocation guide: - 1 mark = one correct fact or identification - 2-3 marks = fact + explanation/justification - 5+ marks = structured answer with multiple components

11.5 Key Formulas Quick Reference

Formula	Purpose
`Speed-up = Serial_Time / Parallel_Time`	Measuring parallelism effectiveness
`p_fp = (1 - e^(-kn/m))^k`	Bloom filter false positive probability
`k_opt = (m/n) × ln(2)`	Optimal number of hash functions
`m = -(n × ln(p_fp)) / (ln(2))^2`	Required bits for target FP rate
`R + W > N`	Strong consistency condition (quorum)
`Quorum = ⌊N/2⌋ + 1`	Quorum size for replication factor N

11.6 Technology Timeline

```
1984  NFS developed (Sun Microsystems)
1998  Google incorporated
2000  Google GFS enters production
2002  Google MapReduce enters production
2003  Google reveals GFS paper
2004  Google reveals MapReduce paper
2005  Google develops BigTable
2006  Yahoo releases Hadoop (open-source MapReduce)
2008  Powerset releases HBase (open-source BigTable)
2009  Spark project starts (UC Berkeley)
2010  Spark open-sourced
2014  Apache Spark officially released
```

Quick Reference: Topic → Section Mapping

Exam Topic	Section
5 V's of Big Data	1.2
Hardware bottlenecks	1.3
Batch vs Stream	1.5, 3.12
Vertical vs Horizontal scaling	1.6
Work distribution challenges	2.1
MapReduce paradigm	2.2
Hadoop architecture	2.3
Hadoop execution (23 steps)	2.4
Combiners, partitioners	2.6
Issues with Hadoop	2.7
Spark vs Hadoop	3.2
RDDs, transformations, actions	3.3, 3.7, 3.8
Lazy evaluation & DAGs	3.4, 3.5
Spark architecture	3.6
Broadcast vars & accumulators	3.10
HDD vs SSD	4.1
NFS architecture & limitations	4.3
HDFS architecture	4.4
HDFS block size rationale	4.5
HDFS read/write pipelines	4.6, 4.7
HDFS replication policy	4.8
HDFS recovery (block states)	4.9
CAP theorem	5.1, 5.2
ACID vs BASE	5.3
LSM Trees	6.3-6.6
Bloom Filters	6.7-6.9
BigTable data model	7.2
HBase architecture	7.5
ZooKeeper	8.1, 8.2

Exam Topic	Section
DHT & consistent hashing	8.3
Cassandra architecture	9.1-9.3
Consistency levels & quorum	9.4, 9.5
Cassandra vs HBase	9.6
YARN, Mesos, Kubernetes	10.1
Containers vs VMs	10.2
GDPR & ethics	10.4

Generated from COMPSCI4064/5088 lecture materials (Weeks 1-10) and past papers (2015-2023). University of Glasgow.